

Entwurf digitaler Systeme

3. Schaltalgebra und Boolesche Funktionen

3. Schaltalgebra und Boolesche Algebra

3.1 Begriffsbestimmung

Die *Boolesche Algebra* stellt die Basis für die Analyse und Synthese digitaler Schaltungen dar. Die von dem englischen Mathematiker G. Boole (1815-1864) benannte Algebra der formalen Logik wird jedoch nicht in ihrer strengen Form verwendet, sondern ist an zweiwertige Schaltgrößen angepasst – daher der Name *Schaltalgebra*.

Die für die Digitaltechnik instrumentierbare Struktur der Schaltalgebra ist wie folgt definiert:

$$SA := (\{0,1\}, +, \cdot, ^-, 0, 1)$$

Wir haben $\{0, 1\}$ als Menge der Variablen 0 und 1, die verwendet werden, zu interpretieren. **Achtung:** Die Symbole 0 und 1 der Menge $\{0, 1\}$ stellen *nicht* (!) die ganzen Zahlen 0 und 1 dar.

3.1 Begriffsbestimmung

Die unären Operatoren „ $+$ “, „ $\cdot$ “ und „ $-$ “ stellen die Operation der Vereinigung (Disjunktion), des Durchschnitts (Konjunktion) und der Komplementbildung dar. Diese Symbole werden in der Schaltalgebra als ODER-, UND- sowie NICHT-Funktionen bezeichnet. Die Operationstabellen, auch *Wahrheitstabellen* genannt, für die Schaltalgebra lauten:

A	B	$+$	$\cdot$
0	0	0	0
0	1	1	0
1	0	1	0
1	1	1	1

Disjunktion Konjunktion
 ODER UND

A	$-$
0	1
1	0

Komplement
 NICHT

3.1 Begriffsbestimmung

Die Tabellen geben an, unter welchen Bedingungen eine boolesche Verknüpfung in *positiver Logik* *wahr* = 1 oder *falsch* = 0 ist. Diese Tabellen können ebenso in *negativer Logik* formuliert werden.

Es sei ausdrücklich darauf hingewiesen, dass neben den Operationssymbolen $(+, \bullet, -)$ weitere gebräuchlich sind.

Zu nennen sind hier:

$(\vee, \wedge, \neg)$ nach DIN 66000

sowie $(\vee, \wedge, /)$ $(+, \&, /)$ $(+, \&, ')$ $(\#, \&, !)$

3.1 Begriffsbestimmung

Wir werden die erst genannte Festlegung nutzen, da diese auch im Wesentlichen in Simulations-, Synthese- und Analysesoftware Verwendung findet.

Auch wird die UND-Operation in vielen Fällen nicht aufgeschrieben, da meist keine Verwechslungen zu erwarten sind.

Weiterhin gilt, dass die Konjunktion Vorrang vor allen anderen Verknüpfungen hat. Das Negationszeichen übt die stärkste Bindung auf eine (oder mehrere) Variable aus.

3.2 Das Operationensystem der Booleschen Algebra

Der Umgang mit der Booleschen Algebra erfordert Übung, denn das formale Anwenden führt in der Regel nicht zum Erfolg. Erst wenn Sie klare Vorstellungen von Ihrer angestrebten Lösung haben, sind Sie in der Lage die geeigneten Regeln auszuwählen. Dennoch sollten Sie sich nicht davon abschrecken lassen, das Operationensystem zu beherrschen. Es ist notwendig.

Es werden die Variablen $A, B, C \in \{0, 1\}$ verwendet.

Kommutative Beziehungen

$$A + B = B + A$$

$$A \cdot B = B \cdot A$$

3.2 Das Operationensystem der Booleschen Algebra

Assoziative Beziehungen

$$(C + B) + A = C + (B + A)$$

$$(C \cdot B) \cdot A = C \cdot (B \cdot A)$$

Distributive Beziehungen

$$C + (B \cdot A) = (C + B) \cdot (C + A)$$

$$C \cdot (B + A) = (C \cdot B) + (C \cdot A)$$

3.2 Das Operationensystem der Booleschen Algebra

Elementoperationen

Identität $A + 0 = 0 + A = A$ und $A \cdot 1 = 1 \cdot A = A$

1/0-Element $A + 1 = 1 + A = 1$ und $A \cdot 0 = 0 \cdot A = 0$

Idempotenz $A + A = A$ und $A \cdot A = A$ (Tautologie)

Komplement $A + \overline{A} = 1$ und $A \cdot \overline{A} = 0$ und $\overline{\overline{A}} = A$

3.2 Das Operationensystem der Booleschen Algebra

Absorption

$$B \cdot (B + A) = B$$

$$B + (B \cdot A) = B$$

$$B \cdot (\overline{B} + A) = B \cdot A$$

$$B + (\overline{B} \cdot A) = B + A$$

De Morgan Regeln

$$\overline{(A + B)} = \overline{A} \cdot \overline{B} \quad \text{und} \quad \overline{(A \cdot B)} = \overline{A} + \overline{B}$$

generalisiert $\overline{(A + B + C + \dots)} = \overline{A} \cdot \overline{B} \cdot \overline{C} \cdot \dots$

$$\overline{(A \cdot B \cdot C \cdot \dots)} = \overline{A} + \overline{B} + \overline{C} + \dots$$

3.2 Das Operationensystem der Booleschen Algebra

Konsensus

$$A \cdot B + \overline{A} \cdot C + B \cdot C = A \cdot B + \overline{A} \cdot C$$

$$(A + B) \cdot (\overline{A} + C) \cdot (B + C) = (A + B) \cdot (\overline{A} + C)$$

Die in den Regeln wiedergegebenen Beziehungen der Schaltalgebra stellen zunächst eine Aussage über die Gleichheit der rechten und linken Seite der verschiedenen Gleichungen dar. Dieser Sachverhalt gilt für sämtliche Variablenbelegungen der jeweiligen Gleichungen. Mit Hilfe der Regeln lassen sich Gleichungen der Schaltalgebra systematisch umformen – mit dem primären Ziel der Gleichungsvereinfachung auf *minimalen* Aufwand hin.

3.2 Das Operationensystem der Booleschen Algebra

Eine Boolesche Funktion $Z = f(A, B, C, \dots)$ stellt, wie auch jede andere Funktion, eine Abbildungsvorschrift dar.

Ausgehend von einer binären Eingangsbelegung (Bündel, Vektor) $x = (x_0, x_1, \dots, x_{N-1})$ mit N Einträgen $x_i \in \{0, 1\}$, lässt sich eine eindeutige Ausgangsfunktion $Z = f(x_0, x_1, \dots, x_{N-1})$ auf die Werte $\{0, 1, \times\}$ abbilden. Dabei kann der so genannte *Don't-Care-Wert* „X“ oder „–“ jeweils beliebig auf 0 oder 1 gesetzt werden.

3.3 Normalformen der Booleschen Algebra

Allgemeine Form der *Wahrheitstabelle* Boolescher Funktionen

x_{N-1}	...	x_1	x_0	$f(x_{N-1}, x_{N-2}, \dots, x_0)$
0	...	0	0	$f(0, 0, \dots, 0)$
0	...	0	1	$f(0, 0, \dots, 1)$
$\vdots$	...	$\vdots$	$\vdots$	$\vdots$
1	...	1	1	$f(1, 1, \dots, 1)$

3.3 Normalformen der Booleschen Algebra

Aus dieser Tabelle lässt sich die Boolesche Funktion in der allgemeinen Form ablesen. Sie lautet:

$$\begin{aligned}
 Z &= f(x_{N-1}, x_{N-2}, \dots, x_0) = \dots \\
 &\dots = [\overline{x_{N-1}} \cdot \overline{x_{N-2}} \cdot \dots \cdot \overline{x_0}] \cdot f(0, 0, \dots, 0) + \dots \\
 &\dots + [\overline{x_{N-1}} \cdot \overline{x_{N-2}} \cdot \dots \cdot x_0] \cdot f(0, 0, \dots, 1) + \dots \\
 &\vdots \\
 &\dots + [x_{N-1} \cdot x_{N-2} \cdot \dots \cdot x_0] \cdot f(1, 1, \dots, 1).
 \end{aligned}$$

Disjunktion

Konjunktion

3.3 Normalformen der Booleschen Algebra

Die Wahrheitstabelle gibt den Zusammenhang zwischen den unabhängigen Eingangsvariablen und den abhängigen Ausgangsvariablen wieder.

Die Eingangsvariablen werden in allen Kombinationsmöglichkeiten dargestellt.

Für N Eingangsvariable gibt es also 2^N Kombinationen in der Wahrheitstabelle.

Beispiel I: $N = 2$ Eingangsvariable A, B und Ausgangsvariable Z

B	A	Z
0	0	0
0	1	0
1	0	0
1	1	1

$$Z = [\overline{A} \cdot \overline{B}] \cdot 0 + [A \cdot \overline{B}] \cdot 0 + [\overline{A} \cdot B] \cdot 0 + [A \cdot B] \cdot 1 = \dots$$

$$\dots = A \cdot B.$$

3.3 Normalformen der Booleschen Algebra

Beispiel II:

C	B	A	Z
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

$$Z = \overline{A}\overline{B}\overline{C} \cdot 0 + \overline{A}\overline{B}C \cdot 1 + \overline{A}B\overline{C} \cdot 1 + A\overline{B}\overline{C} \cdot 0 + \overline{A}BC \cdot 1 + \dots$$

$$\dots + A\overline{B}C \cdot 0 + \overline{A}BC \cdot 0 + ABC \cdot 1.$$

vereinfacht

$$Z = \overline{A}\overline{B}C + \overline{A}B\overline{C} + \overline{A}BC + ABC.$$



Sie sehen, es sind nur die oben fett markierten Konjunktionen zu berücksichtigen.

3.3 Normalformen der Booleschen Algebra

Einige Begriffe:

$f(x_{N-1}, \dots, x_0)$ heißt *vollständig definiert*, falls der Wertebereich ausschließlich aus der Menge $\{0,1\}$ besteht.

$f(x_{N-1}, \dots, x_0)$ heißt *unvollständig definiert*, falls der Wertebereich aus der Menge $\{0,1,x\}$ besteht.

$(A \cdot \bar{B} \cdot \bar{C} \cdot D \cdot E \dots)$ heißt *Funktionsausdruck* oder *Term*.

$(A \cdot \bar{B} \cdot \bar{C} \cdot D \cdot E \dots)$ eine Variable *innerhalb* eines Terms wird *Literal* genannt.

Beispielsweise besitzt die Funktion $f(A, B, C) = A\bar{B}\bar{C} + \bar{A}BC + A\bar{B}C$ drei Variablen (A,B,C), aber zwölf *Literale*.

3.3 Normalformen der Booleschen Algebra

Wir wollen zu unserer allgemeinen Form der Booleschen Funktion zurückkehren. Die oben genannte Schreibweise einer Booleschen Funktion wird als *Kanonische* disjunktive Normalform* (KDNF) bezeichnet.

■ Definition 3.1 (Kanonische disjunktive Normalform)

Die Kanonische disjunktive Normalform (KDNF) einer Booleschen Funktion ist die disjunktive Verknüpfung einer Anzahl M von Konjunktion K_i (Termen).

$$f(\mathbf{x}) = K_0 + K_1 + \dots + K_i + \dots + K_{M-1}$$

Dabei stellt jede Konjunktion die Verknüpfung aller N Variablen in negierter oder nicht negierter Form dar. Eine derartige Konjunktion wird als Minterm bezeichnet, da sie nur für eine spezifische Wertekombination ihrer N Variablen einen Ausgangswert 1 liefert.

*)Das **kanonische Format** bezeichnet eine *allgemeingültige* und *eindeutige* Bezeichnung eines Datensatzes oder einer Gleichung.

3.3 Normalformen der Booleschen Algebra

Wir betrachten nochmals die uns schon bekannte Tabelle:

C	B	A	Z
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

$Z = \overline{A}\overline{B}\overline{C} \cdot 0 + \overline{A}\overline{B}C \cdot 1 + \overline{A}B\overline{C} \cdot 1 + \overline{A}BC \cdot 0 + A\overline{B}\overline{C} \cdot 1 + \dots$
 $\dots + A\overline{B}C \cdot 0 + ABC \cdot 0 + ABC \cdot 1.$

Es werden nun ausschließlich die „Nullen“ zur Generation einer Beschreibungsform verwendet:

$f(A, B, C) = \overline{Z} = \overline{A}\overline{B}C + \overline{A}B\overline{C} + A\overline{B}\overline{C} + \overline{A}B\overline{C}.$

3.3 Normalformen der Booleschen Algebra

Werden auf die obige Gleichung nun einige der bekannten Regeln angewendet, erhält man:

$$Z = \overline{\overline{A}\overline{B}\overline{C}} + \overline{A\overline{B}\overline{C}} + \overline{A\overline{B}C} + \overline{A\overline{B}C} = \dots$$

$$\dots = (\overline{\overline{A}\overline{B}\overline{C}}) \cdot (\overline{A\overline{B}\overline{C}}) \cdot (\overline{A\overline{B}C}) \cdot (\overline{A\overline{B}C}) = \dots$$

$$\dots = (\overline{\overline{A} + \overline{B} + \overline{C}}) \cdot (\overline{\overline{A} + \overline{B} + \overline{C}}) \cdot (\overline{\overline{A} + \overline{B} + \overline{C}}) \cdot (\overline{\overline{A} + \overline{B} + \overline{C}}) = \dots$$

$$\dots = (A + B + C) \cdot (\overline{A} + \overline{B} + C) \cdot (\overline{A} + B + \overline{C}) \cdot (A + \overline{B} + \overline{C}).$$

Diese Schreibweise einer Booleschen Funktion wird als *kanonische konjunktive Normalform* (KKNF) bezeichnet.

3.3 Normalformen der Booleschen Algebra

■ Definition 3.2 (Kanonische konjunktive Normalform)

Die Kanonische konjunktive Normalform (KKNF) einer Booleschen Funktion ist die konjunktive Verknüpfung einer Anzahl M von Disjunktion D_i (Termen).

$$f(\mathbf{x}) = D_0 \cdot D_1 \cdot \dots \cdot D_i \cdot \dots \cdot D_{M-1}$$

Dabei stellt jede Disjunktion die Verknüpfung aller N Variablen in negierter oder nicht negierter Form dar. Eine derartige Disjunktion wird als Maxterm bezeichnet, da sie nur für eine spezifische Wertekombination ihrer N Variablen einen Ausgangswert 0 liefert.

3.3 Normalformen der Booleschen Algebra

- Vielfach lassen sich die Minterme und Maxterme mit Hilfe der Operationsregeln weiter vereinfachen.
- In diesen Fällen enthalten die Terme nicht alle vorkommenden Variablen – die Terme sind dann nicht mehr kanonisch.
- Auch für diese reduzierten Formen werden entsprechende Bezeichnungen und Definitionen verwendet.

3.3 Normalformen der Booleschen Algebra

■ Definition 3.3 (Disjunktive Normalform)

Die disjunktive Normalform (DNF) einer Booleschen Funktion ist die disjunktive Verknüpfung einer Anzahl M von Konjunktionen K_i (Termen).

$$f(\mathbf{x}) = K_0 + K_1 + \dots + K_i + \dots + K_{M-1}$$

Dabei stellt jede Konjunktion die Verknüpfung einer einzigen, mehrerer oder aller N Variablen in negierter oder nicht negierter Form dar.

3.3 Normalformen der Booleschen Algebra

■ Definition 3.4 (Konjunktive Normalform)

Die konjunktive Normalform (KNF) einer Booleschen Funktion ist die konjunktive Verknüpfung einer Anzahl M von Disjunktionen D_i (Termen).

$$f(\mathbf{x}) = D_0 \cdot D_1 \cdot \dots \cdot D_i \cdot \dots \cdot D_{M-1}$$

Dabei stellt jede Disjunktion die Verknüpfung einer einzigen, mehrerer oder aller N Variablen in negierter oder nicht negierter Form dar.

3.3 Normalformen der Booleschen Algebra

- Mit Hilfe der Definitionen 3.1, 3.2, 3.3 und 3.4 konnte gezeigt werden, dass die logischen Funktionen UND, ODER sowie Negation ausreichen, um *jede beliebige* Boolesche Funktion darzustellen.
- Für jede Boolesche Funktion existiert, abgesehen von Vertauschungen der Variablen, wegen der Kommutativität von Konjunktion und Disjunktion, genau *eine* KDNF und genau *eine* KKNF.
- Es kann *mehrere* DNF und KNF für eine Boolesche Funktion geben.
- Die disjunktive Normalform wird auch als *Summe von Produkten* (*Sum-of-Products, SOP*) bezeichnet. Die konjunktive Normalform nennt man häufig *Produkt von Summen* (*Product-of-Sums, POS*). Diese Begriffe haben sich insbesondere im Bereich der programmierbaren Bausteine durchgesetzt. Sie finden die Bezeichnungen üblicherweise in den entsprechenden Datenblättern.

3.4 Symbolische Funktionsbeschreibung durch Gatter

- Für die Gatterrepräsentation verschiedener Grundfunktionen sind Symbole festgelegt, die sich jedoch weltweit nicht einheitlich darstellen. In Europa verwendet man Symbole nach **IEC-Standard 60617-12** (*International Electrotechnical Commission*), bzw. nach **DIN 40900** (*Deutsches Institut für Normung*). Dieser ist auch vom **IEEE** (*Institute for Electrical and Electronics Engineers*) übernommen worden. Daneben sind die US-amerikanischen Symbole nach **ANSI-Standard** (*American National Standards Institute*) gebräuchlich.
- In dieser Vorlesung, Übung und Praktikum werden bewusst **IEC- und ANSI-Symbole** verwendet, da in der Praxis beide Varianten Verwendung finden. Deshalb ist es notwendig mit beiden Symbolfamilien umgehen zu können.

3.4.1 Elementartypen

Der *Puffer* ist die erste einfachste Grundfunktion.

Mit $N = 1$ Eingangsvariablen. $2^N = 2$ Kombinationen.

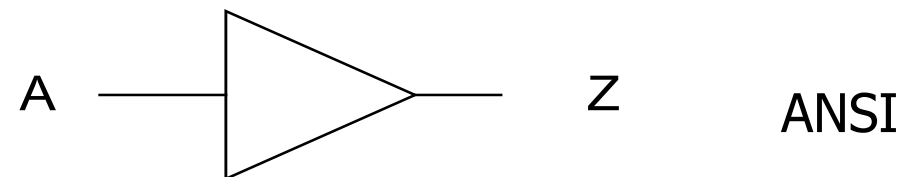
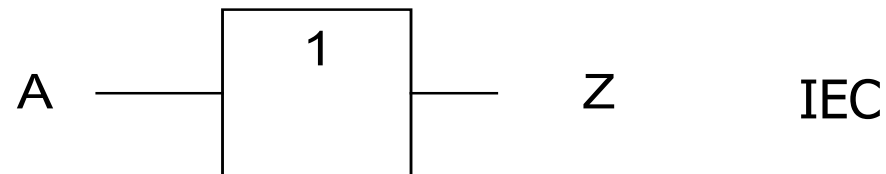
Funktion: $Z = A$

Wahrheitstabelle

A	Z
0	0
1	1

3.4.1 Elementartypen

Schaltsymbole des Puffers nach IEC- und ANSI-Standard.



3.4.1 Elementartypen

Die *Negation* ist die zweite einfachste Grundfunktion.

Mit $N = 1$ Eingangsvariablen. $2^N = 2$ Kombinationen.

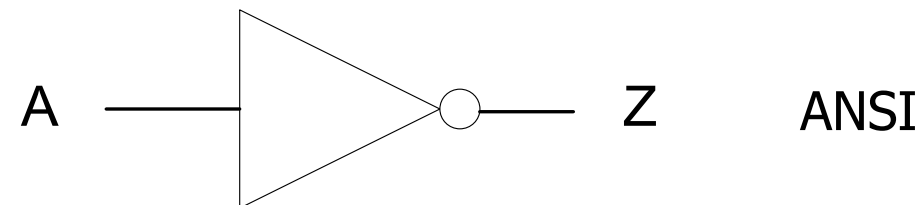
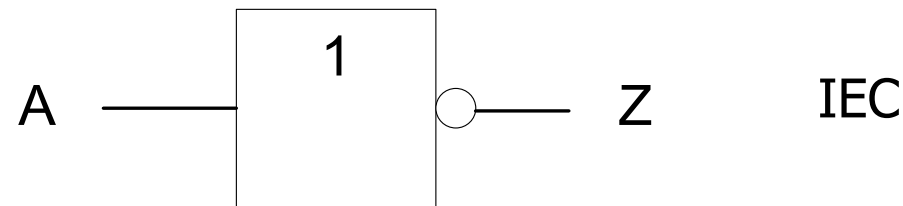
Funktion: $Z = \overline{A} \equiv / A \equiv \neg A \equiv A' \equiv !A$

Wahrheitstabelle

A	Z
0	1
1	0

3.4.1 Elementartypen

Schaltsymbole der Negation nach IEC- und ANSI-Standard.



Hinweis:

Der Punkt stellt symbolisch die Negation dar. Es ist zu berücksichtigen, dass der Punkt auch an einem oder mehreren Eingängen als Negation verwendet werden kann.

3.4.1 Elementartypen

Die **UND** -Funktion (Konjunktion); UND-Gatter, AND-Gate

Mit $N = 2$ Eingangsvariablen. $2^N = 4$ Kombinationen.

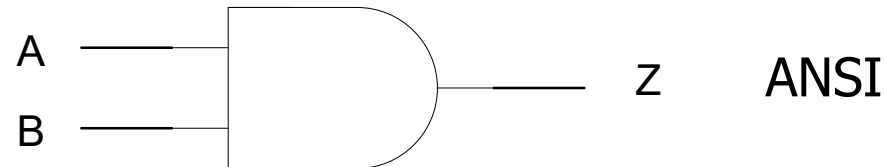
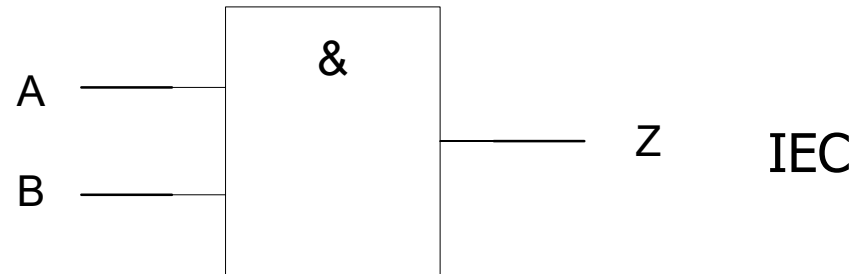
Funktion: $Z = A \cdot B \equiv AB \equiv A \wedge B \equiv A \& B$

Wahrheitstabelle

B	A	Z
0	0	0
0	1	0
1	0	0
1	1	1

3.4.1 Elementartypen

Schaltsymbole der UND-Funktion nach IEC- und ANSI-Standard.



Hinweis:

Für diesen Gattertyp können auch mehrere Eingänge (3, 4, 5, ...) definiert werden.

3.4.1 Elementartypen

Die *ODER* -Funktion (Disjunktion); ODER-Gatter, OR-Gate

Mit $N = 2$ Eingangsvariablen. $2^N = 4$ Kombinationen.

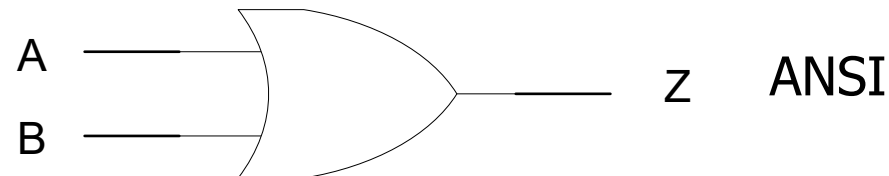
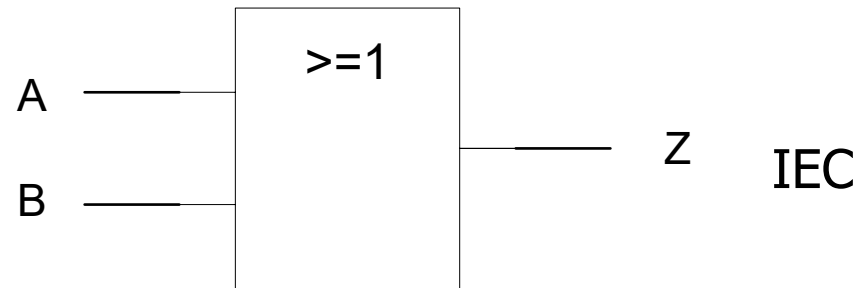
Funktion: $Z = A + B \equiv A \vee B \equiv A \# B$

Wahrheitstabelle

B	A	Z
0	0	0
0	1	1
1	0	1
1	1	1

3.4.1 Elementartypen

Schaltsymbole der ODER-Funktion nach IEC- und ANSI-Standard.



Hinweis:

Für diesen Gattertyp können auch mehrere Eingänge (3, 4, 5, ...) definiert werden.

3.4.2 Abgeleitete Gattertypen

- Wie aus Kapitel 3.3 bekannt ist, können mit den Elementarfunktionen und somit den Gattertypen alle logischen Funktionen realisiert werden.
- In einer Vielzahl von Anwendungsfällen erweist es sich jedoch als nützlich, weitere, aus den Elementartypen abgeleitete Funktionen zu verwenden.
- Sie haben erkannt, dass unter Verwendung von Negationen in Verbindung mit den De Morgan Regeln, aus der disjunktiven Normalform die konjunktive Normalform erzeugt werden kann. Dieser Sachverhalt lässt sich auch unter Verwendung abgeleiteter Gattertypen formulieren.

3.4.2 Abgeleitete Gattertypen

Die **NAND** –Funktion; NAND-Gatter, NAND-Gate

Mit $N = 2$ Eingangsvariablen. $2^N = 4$ Kombinationen.

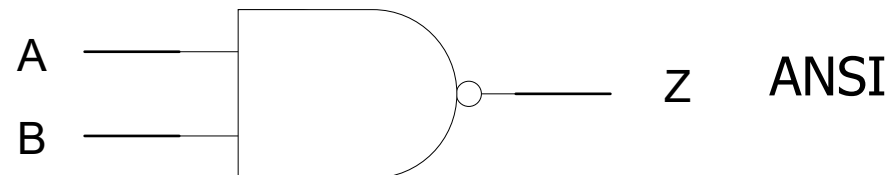
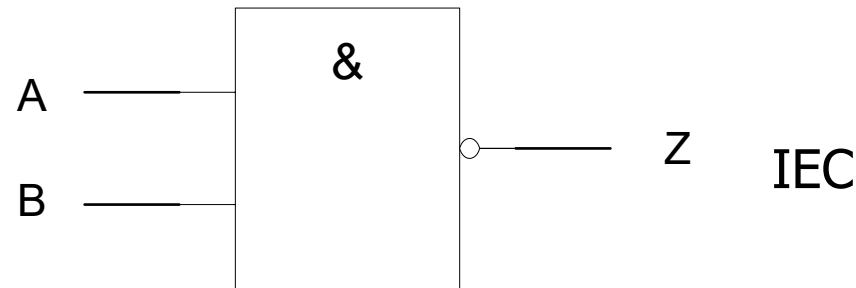
Funktion: $Z = \overline{A \cdot B} \equiv \overline{AB} \equiv \overline{A \wedge B} \equiv \overline{A \& B}$

Wahrheitstabelle

B	A	Z
0	0	1
0	1	1
1	0	1
1	1	0

3.4.2 Abgeleitete Gattertypen

Schaltsymbole der NAND-Funktion nach IEC- und ANSI-Standard.



Hinweis:

Für diesen Gattertyp können auch mehrere Eingänge (3, 4, 5, ...) definiert werden.

3.4.2 Abgeleitete Gattertypen

Die **NOR** –Funktion; NOR-Gatter, NOR-Gate

Mit $N = 2$ Eingangsvariablen. $2^N = 4$ Kombinationen.

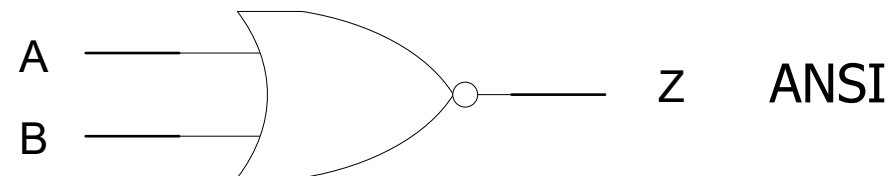
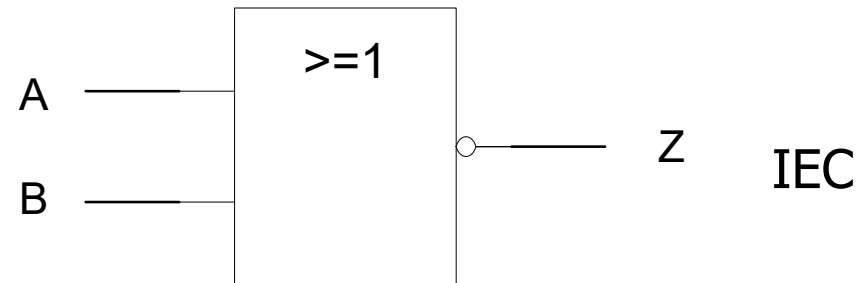
Funktion: $Z = \overline{A + B} \equiv \overline{A \vee B} \equiv \overline{A \# B}$

Wahrheitstabelle

B	A	Z
0	0	1
0	1	0
1	0	0
1	1	0

3.4.2 Abgeleitete Gattertypen

Schaltsymbole der NOR-Funktion nach IEC- und ANSI-Standard.



Hinweis:

Für diesen Gattertyp können auch mehrere Eingänge (3, 4, 5, ...) definiert werden.

3.4.2 Abgeleitete Gattertypen

Die **XOR** -Funktion (andere Bezeichnung: Exklusive-Oder bzw. Antivalenz);
XOR-Gatter, XOR-Gate

Mit $N = 2$ Eingangsvariablen. $2^N = 4$ Kombinationen.

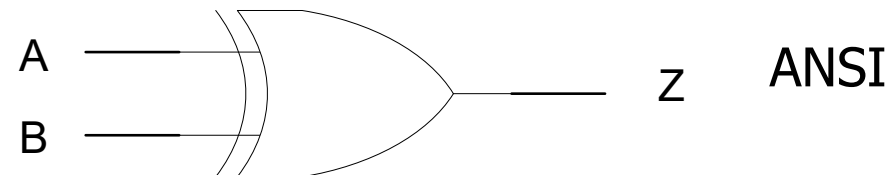
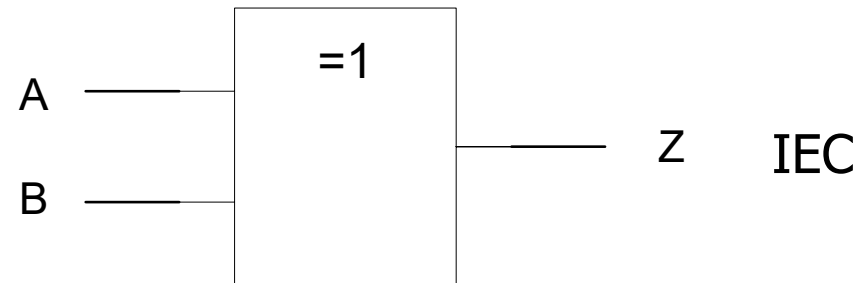
Funktion: $Z = A \cdot \overline{B} + \overline{A} \cdot B = A \nabla B = A \oplus B = A \neq B$

Wahrheitstabelle

B	A	Z
0	0	0
0	1	1
1	0	1
1	1	0

3.4.2 Abgeleitete Gattertypen

Schaltsymbole der XOR-Funktion nach IEC- und ANSI-Standard.



Hinweis:

Streng genommen gelten die Gattersymbole nur für zwei Eingänge. Wir werden später auf einige Rechenregeln für eine Anzahl von Eingangsvariablen > 2 zurückkommen.

3.4.2 Abgeleitete Gattertypen

Antivalenz-Rechenregeln:

$$0 \oplus A = A,$$

$$1 \oplus A = \overline{A},$$

$$A \oplus A = 0,$$

$$A \oplus \overline{A} = 1,$$

$$A \oplus B = B \oplus A \quad (\text{Kommutativitat}),$$

$$(A \oplus B) \oplus C = A \oplus (B \oplus C) \quad (\text{Assoziativitat}).$$

3.4.2 Abgeleitete Gattertypen

Die *XNOR* –Funktion (Äquivalenz); XNOR-Gatter, XNOR-Gate

Mit $N = 2$ Eingangsvariablen. $2^N = 4$ Kombinationen.

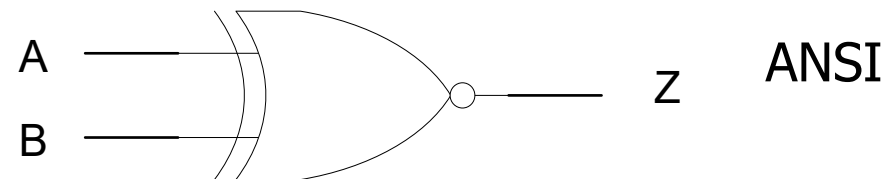
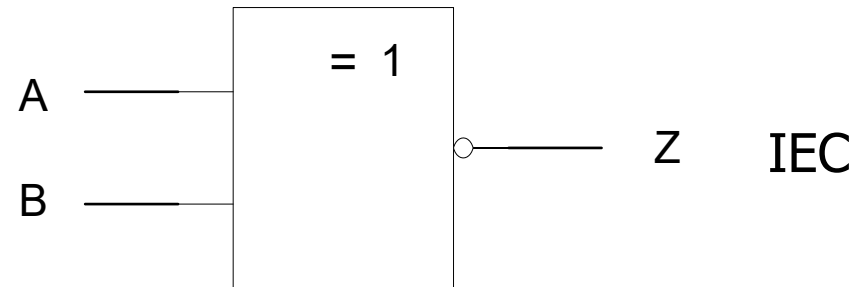
Funktion: $Z = \overline{B} \cdot \overline{A} + A \cdot B = A \sim B = A \equiv B = \overline{A \oplus B}$

Wahrheitstabelle

B	A	Z
0	0	1
0	1	0
1	0	0
1	1	1

3.4.2 Abgeleitete Gattertypen

Schaltsymbole der XNOR-Funktion nach IEC- und ANSI-Standard.



Hinweis:

Streng genommen gelten die Gattersymbole nur für zwei Eingänge. Wir werden später auf einige Rechenregeln für eine Anzahl von Eingangsvariablen > 2 zurückkommen.

3.4.2 Abgeleitete Gattertypen

Äquivalenz-Rechenregeln:

$$0 \sim A = \overline{A},$$

$$1 \sim A = A,$$

$$A \sim A = 1,$$

$$A \sim \overline{A} = 0,$$

$$A \sim B = B \sim A \quad (\text{Kommutativität}),$$

$$(A \sim B) \sim C = A \sim (B \sim C) \quad (\text{Assoziativität}).$$

Spezielle weitere Rechenregeln finden Sie in der Literatur.

3.4.3 Technische Logikrealisierung

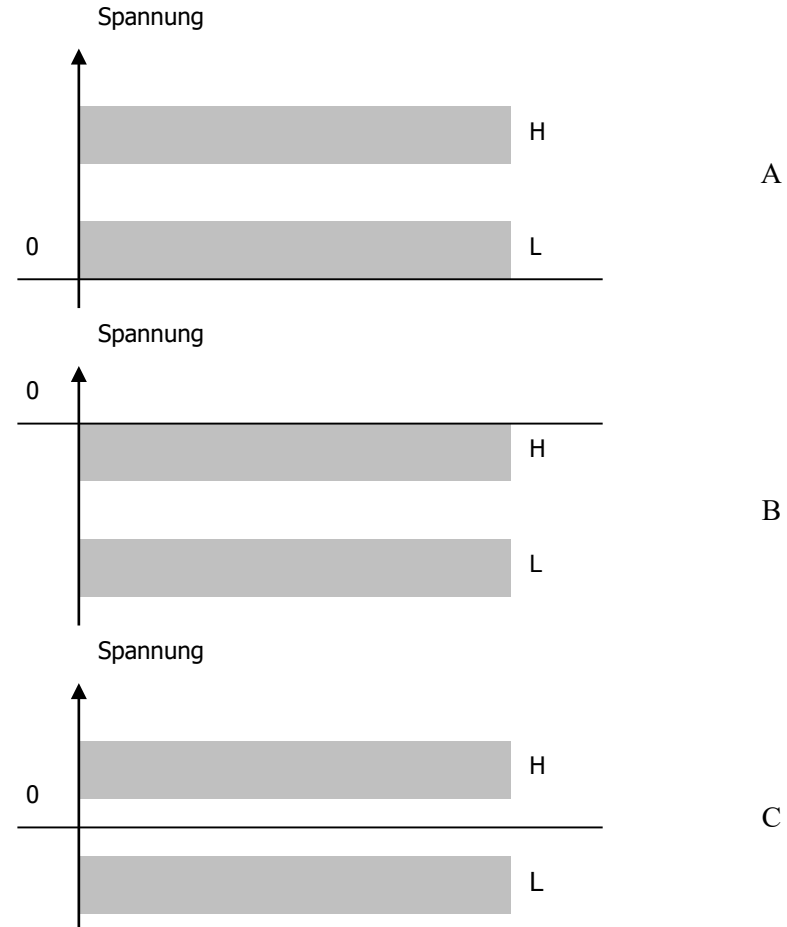
- Man unterscheidet zwischen Logik-Zustand und Logik-Pegel einer binären Variablen. Der Logik-Zustand beschreibt im Sinne der Booleschen Algebra unterschiedliche Zustände verschiedener Eingangs-, Zwischen-, und Ausgangsvariablen. Die zugeordneten Variablen werden bekanntermaßen mit den Zuständen (bzw. Ziffern) 0 und 1 belegt.
- Demgegenüber beschreibt man das Verhalten einer digitalen Schaltung mit Logik-Pegeln. Sie stellen das physikalische Äquivalent der Logik-Zustände dar. Die Logik-Pegel werden üblicherweise mit *L* (*Low*) und *H* (*High*) bezeichnet.
- Da wir hier elektrische Schaltkreise und Schaltungen betrachten, kommen als Größen immer Spannung bzw. Strom in Betracht. Für verschiedene Logikfamilien sind die Pegelbereiche festgelegt.

3.4.3 Technische Logikrealisierung

- Aufgrund von üblichen Bauteiltoleranzen werden nicht zwei feste Pegel, sondern zwei Bereiche angegeben, die dem L-Pegel und dem H-Pegel entsprechen.
- Dazwischen liegt ein *verbotenes Pegelband*, in dem der logische Zustand nicht definiert ist.
- Da verschiedene Technologien verwendet werden, können die definierten L- und H-Pegel einerseits nur im positiven Spannungsbereich oder nur im negativen Spannungsbereich liegen, andererseits können die Pegel auch im positiven und negativen Spannungsbereich auftreten.

3.4.3 Technische Logikrealisierung

- Beispiele hierzu sind:
 - Transistor-Transistor-Logik (TTL)
 - L: positiv; H: positiv.
 - CMOS-Logik (CMOS)
 - L: positiv; H: positiv.
 - Emitter-gekoppelte Logik (Emitter Coupled Logic, ECL)
 - L: negativ; H: negativ.
 - Leitungstreiber
 - L: negativ; H: positiv.



Verschiedene Logikpegel. A) TTL und CMOS, B) ECL, C) Leitungstreiber.

3.4.3 Technische Logikrealisierung

- Die unterschiedlichen technischen Pegel sind von den Bauteilfamilien und deren technologischer Umsetzung abhängig.
- Die Tendenz geht zu immer niedrigeren Pegeln, damit die Leistungsaufnahme gerade hochkomplexer Bausteine (Moore's Annahme!) in Grenzen gehalten werden kann.
- Während in der TTL-Logik die 5V-Versorgungsspannung üblich ist, werden komplexe CMOS-Bausteine heute mit 3.3V-, 2.5V-, 1.8V-, 1.5V und 1.2V-Versorgungsspannung betrieben. Der Trend geht hier noch weiter abwärts (<1.0 Volt).
- Eine weitere Reduktion der Versorgungsspannung wird in Zukunft notwendig sein (1.0V, 0.8V).

3.4.3 Technische Logikrealisierung

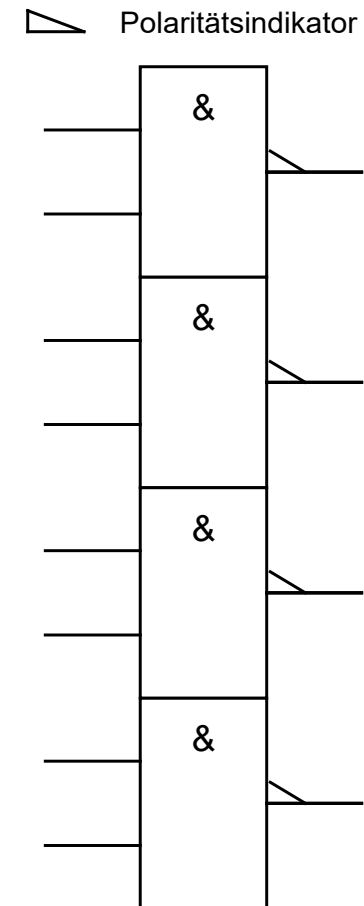
■ Einheitliche Logikvereinbarung

- Für das gesamte digitale System oder zumindest für größere zusammenhängende Teile davon, wird einheitlich der Logik-Zustand und der Logik-Pegel definiert. Man unterscheidet hier zwischen *positiver* und *negativer Logikvereinbarung*.
 - Positive Logikvereinbarung
 - Dem 1-Zustand ist der H-Pegel zugeordnet; dem 0-Zustand ist der L-Pegel zugeordnet.
 - Negative Logikvereinbarung
 - Dem 1-Zustand ist der L-Pegel zugeordnet; dem 0-Zustand ist der H-Pegel zugeordnet.

3.4.3 Technische Logikrealisierung

■ Direkte Logikvereinbarung

- Die direkte Logikvereinbarung bezieht sich auf jedes Gatter (Baustein) einzeln. Innerhalb des Bausteins wird das Boolesche Funktionssystem verwendet, außerhalb das physikalische. Man verwendet an Ein- und Ausgängen einen *Polaritätsindikator*. Fehlt dieser, gilt grundsätzlich die positive Logikvereinbarung.
- Diese Variante der Logikvereinbarung geben Halbleiterhersteller in ihren Datenblättern an, um eine eindeutige Zuordnung für den jeweiligen Baustein herzustellen. Ein NAND-Baustein (z. B. der TTL Baustein SN 74LS00: Vier NAND-Gatter mit jeweils 2 Eingängen in einem Gehäuse) eines Halbleiterherstellers besitzt dann folgendes Aussehen.



3.5 Elementare Logikschaltungen

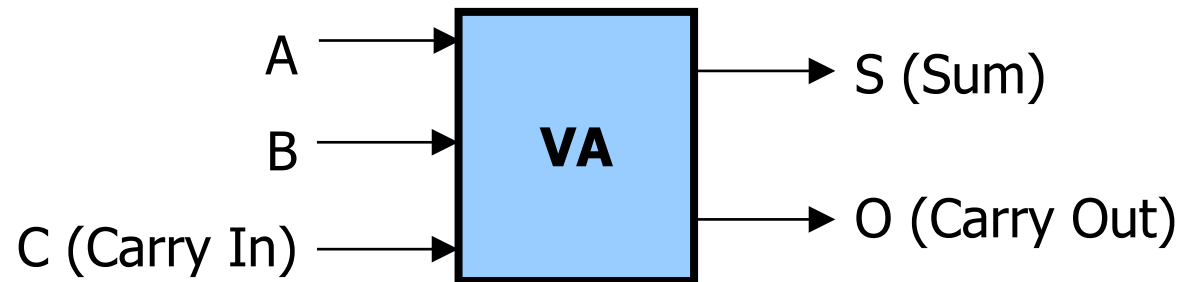
- Zunächst wollen wir uns mit Gatternetzstrukturen, bzw. einfachen kombinatorischen Schaltungen vertraut machen. Ein Gatternetz, auch

Schaltnetz

genannt, das eine bestimmte binäre Funktion realisiert, stellt eine Abbildung der Notationsform einer spezifischen Booleschen Funktion dar.

3.5.1 Einfache Schaltnetze

- Beispiel I: Addierer-Zelle (binärer Volladdierer)



3.5.1 Einfache Schaltnetze

- Die Wahrheitstabelle des Volladdierers lautet:
- Dabei stellen A , B und C (Carry In) die binären Eingangsvariablen und S (Sum) und O (Carry Out) die Ausgangswerte dar. Wir stellen mit Hilfe der Wahrheitstabelle die disjunktive Normalform für die beiden Ausgänge dar.

B	A	C	S	O
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

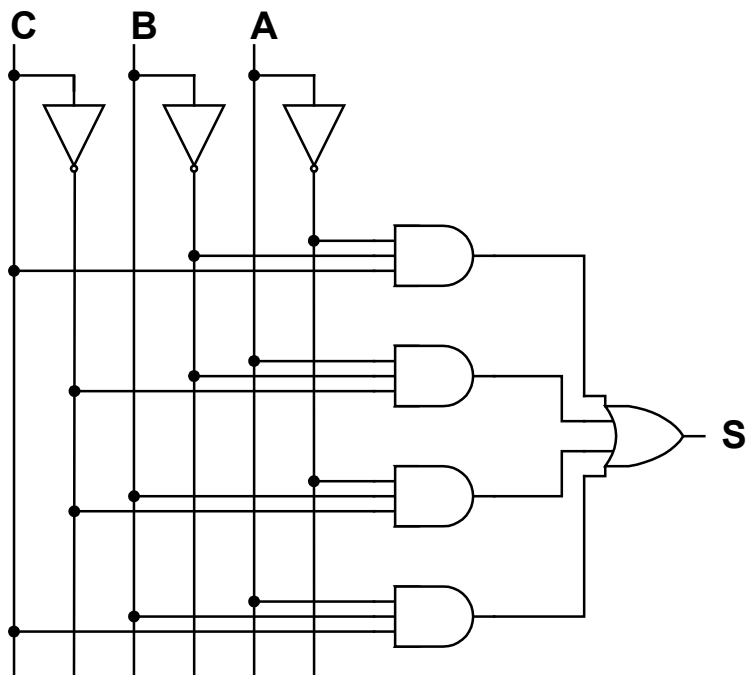
$$S = \bar{A} \cdot \bar{B} \cdot C + A \cdot \bar{B} \cdot \bar{C} + \bar{A} \cdot B \cdot \bar{C} + A \cdot B \cdot C$$

$$O = \bar{A} \cdot B \cdot C + A \cdot \bar{B} \cdot C + A \cdot B \cdot \bar{C} + A \cdot B \cdot C$$

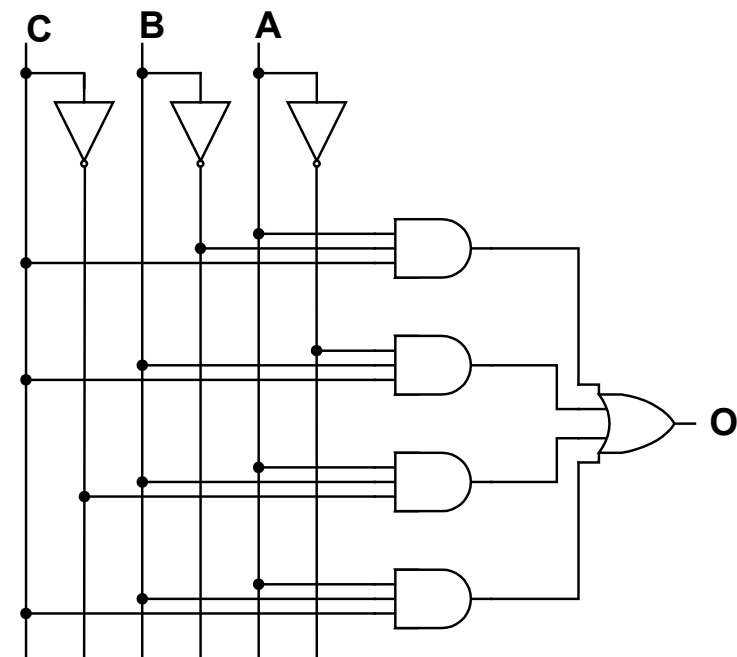
KDNF

3.5.1 Einfache Schaltnetze

- Logische Schaltungen in ANSI-Standard-Darstellung.



Summenausgang eines binären Volladdierers – dargestellt im ANSI-Standard.



Carry Out eines binären Volladdierers – dargestellt im ANSI-Standard.

3.5.1 Einfache Schaltnetze

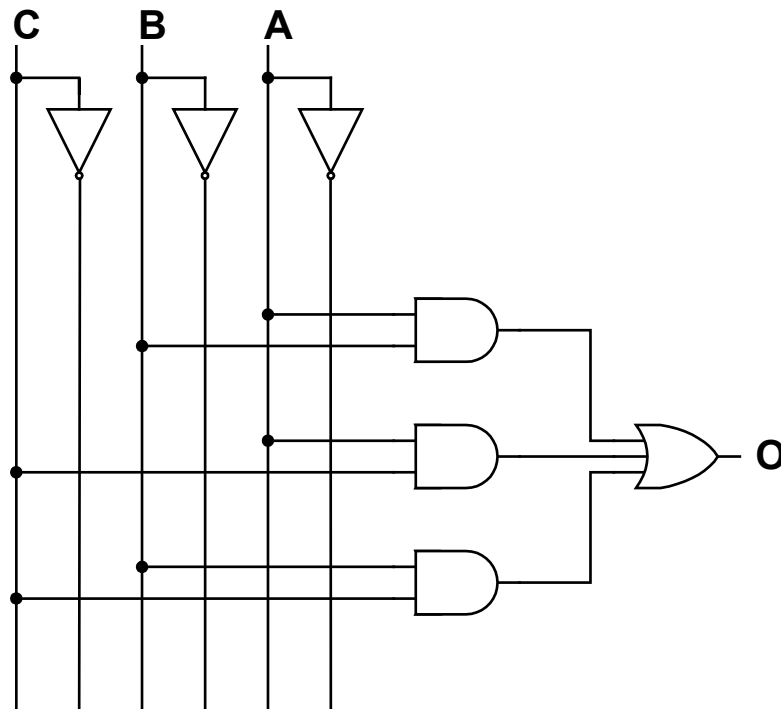
- Nachdem die Schaltungen skizziert sind, stellt sich die Frage, ob diese Schaltungsvarianten die einzig möglichen Realisierungen darstellen. Mit Hilfe unterschiedlicher Regeln könnte es möglich sein, gegebenenfalls Vereinfachungen herbeizuführen, die einfachere Strukturen erlauben. Dieses Vorgehen wollen wir im nächsten Schritt untersuchen. Wir beginnen mit der Carry-Out-Schaltung.

$$\begin{aligned}
 O &= A\overline{B}\overline{C} + ABC + \overline{A}\overline{B}C + \overline{A}BC \\
 &= A\overline{B}\overline{C} + C(AB + \overline{A}\overline{B} + \overline{A}B) \\
 &= A\overline{B}\overline{C} + C[A(B + \overline{B}) + \overline{A}B] = A\overline{B}\overline{C} + C[A + \overline{A}B] \\
 &= A\overline{B}\overline{C} + C[A + B] = A\overline{B}\overline{C} + AC + BC \\
 &= A(\overline{B}\overline{C} + C) + BC = A(B + C) + BC \\
 &= AB + AC + BC
 \end{aligned}$$

Hier sind verschiedene, bekannte Gesetzmäßigkeiten angewendet worden. Mit der Methode des „scharfen Hinsehens“ und einiger Schreibarbeit konnte die Gleichung vereinfacht werden. Leider gelingt dieses nicht bei der Summenschaltung. Versuchen Sie es!

3.5.1 Einfache Schaltnetze

DNF der Carry-Out-Schaltung



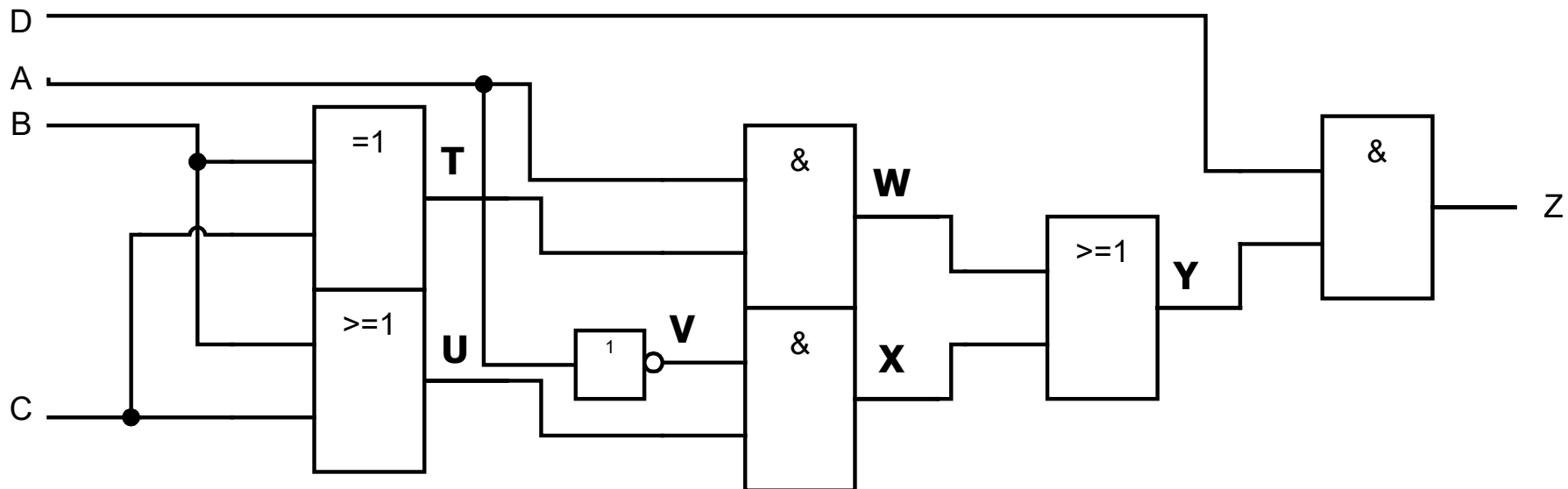
Diese Variante benötigt **drei zweifach-AND** und **ein dreifach-OR**, während die KDNF-Schaltung **vier dreifach-AND** und **ein vierfach-OR** benötigt. Zieht man die Anzahl der Eingänge *aller* Gatter als Vereinfachungsmaß heran, wobei die Inverter nicht gezählt werden sollen, da sie formal auch in die Eingänge gelegt werden können, erhält man:

- Ursprüngliche Variante: 16 Eingänge
- Vereinfachte Variante: 9 Eingänge

3.5.1 Einfache Schaltnetze

Beispiel II:

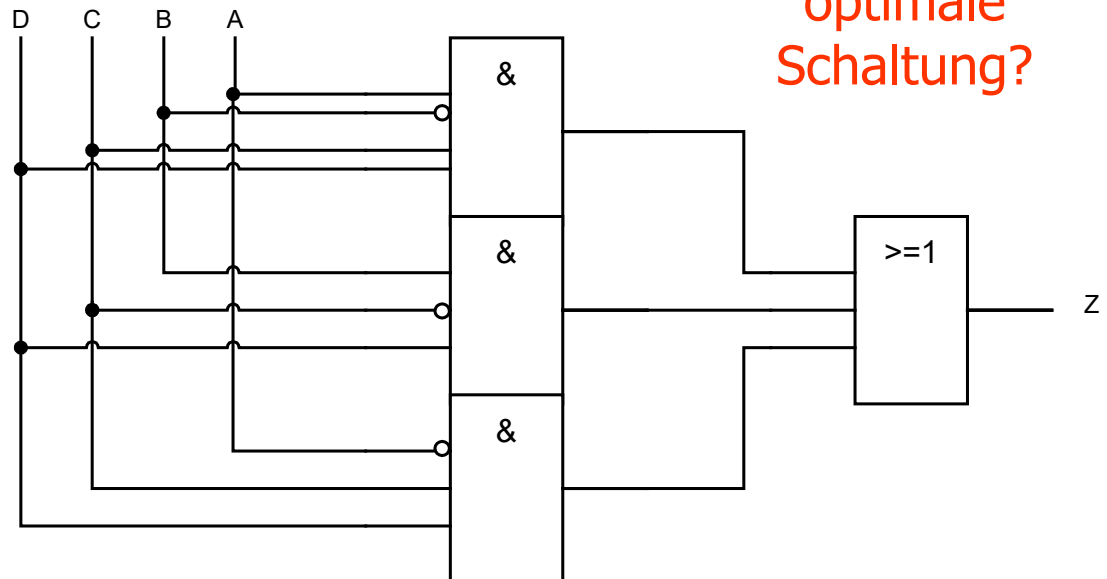
Die beachtliche Aufwandsreduktion zeigt, eine genauere Analyse der vorgegebenen Funktion oder Schaltung in jedem Fall lohnenswert ist. Dieser Sachverhalt soll an einer weiteren Schaltung demonstriert werden. Es werden nun IEC-Symbole verwendet. Gegeben sei folgende Schaltung:



3.5.1 Einfache Schaltnetze

Die Gleichung soll nun in anderer Form mit dem Ziel zusammengefasst werden, eine Vereinfachung herbeizuführen.

$$\begin{aligned}
 Z &= [\bar{A}\bar{B}CD + A\bar{B}\bar{C}D + \bar{A}B\bar{C}D + \bar{A}B\bar{C}\bar{D} + \bar{A}B\bar{C}D] = \dots \\
 &\dots = [\bar{A}\bar{B}CD + (A\bar{B}\bar{C}D + \bar{A}B\bar{C}D) + (\bar{A}B\bar{C}D + \bar{A}B\bar{C}\bar{D})] = \dots \\
 &\dots = [\bar{A}\bar{B}CD + (A + \bar{A})\bar{B}\bar{C}D + (B + \bar{B})\bar{A}\bar{C}D] = \dots \\
 &\dots = [\bar{A}\bar{B}CD + \bar{B}\bar{C}D + \bar{A}\bar{C}D] = \dots?
 \end{aligned}$$



Ist das die optimale Schaltung?

3.5.1 Einfache Schaltnetze

Generell kann gesagt werden, dass die praxisgerechte und zielführende Anwendung der Schaltalgebra und ihren Gesetzen von der **Erfahrung und Intuition** des Anwenders wesentlich abhängt.

Es ist ersichtlich, dass eine Minimierung der Schaltfunktionen **erst** nach einer **Expansion** durchzuführen ist. Erst **im zweiten Schritt** sind **Kürzungs- und Absorptionsregeln** anwendbar.

3.5.2 NAND-Formen

- Im folgenden Abschnitt werden so genannte NAND-Formen betrachtet. Sie sind für den *Schaltungsentwurf* von großem Interesse, da mit dieser Konzeption jede beliebige Schaltung mit Hilfe von NAND-Gattern realisierbar wird.
- Prinzipiell können alle *kombinatorischen Schaltungen* entsprechend transformiert werden.
- Die **NAND-Form (NAF)** einer Funktion enthält ausschließlich NAND-verknüpfte Terme. Die NAF kann durch Mehrfachanwendung der De Morgan Regeln aus KDNF, KKNF oder DNF sowie KNF gebildet werden.
- Die Umsetzung in eine NAF geschieht wie folgt:

3.5.2 NAND-Formen

- Eingangsfunktion: **KDNF** oder **DNF** $f_D(\mathbf{x})$
 - Zweimalige Negation: $\overline{\overline{f_D(\mathbf{x})}}$ führt auf die gesuchte NAF.
 - Das Ergebnis lautet: $Z_{NAF} = \overline{\overline{f_D(\mathbf{x})}}$.

- Eingangsfunktion: **KKNF** oder **KNF** $f_K(\mathbf{x})$
 - Viermalige Negation: $\overline{\overline{\overline{\overline{f_K(\mathbf{x})}}}}$ führt auf die gesuchte NAF.
 - Das Ergebnis lautet: $Z_{NAF} = \overline{\overline{\overline{\overline{f_K(\mathbf{x})}}}}$.

3.5.2 NAND-Formen

Beispiel I: Carry-Out in NAND-Form (DNF)

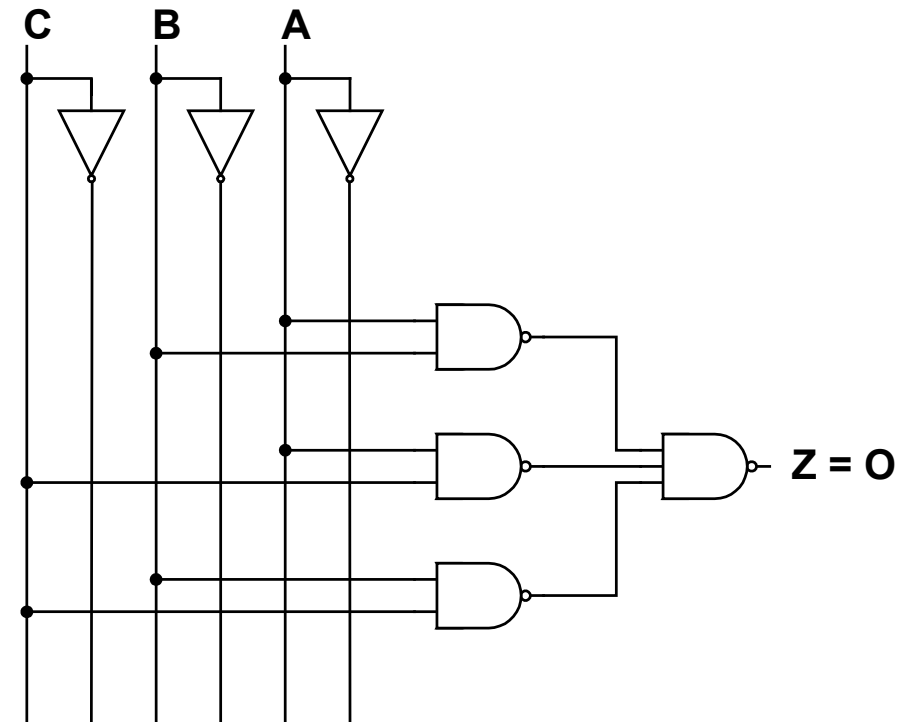
$$f(A, B, C) = AB + AC + BC.$$

$$Z_{NAF} = \overline{\overline{f(A, B, C)}} = \overline{\overline{AB + AC + BC}} = \dots$$

$$\dots = \overline{\overline{AB} \cdot \overline{AC} \cdot \overline{BC}}.$$

De Morgan:

$$\overline{(A \cdot B)} = \overline{A} + \overline{B}$$



3.5.2 NAND-Formen

Beispiel II: KNF

$$f(A, B, C) = (A + B) \cdot (A + C) \cdot (B + C).$$

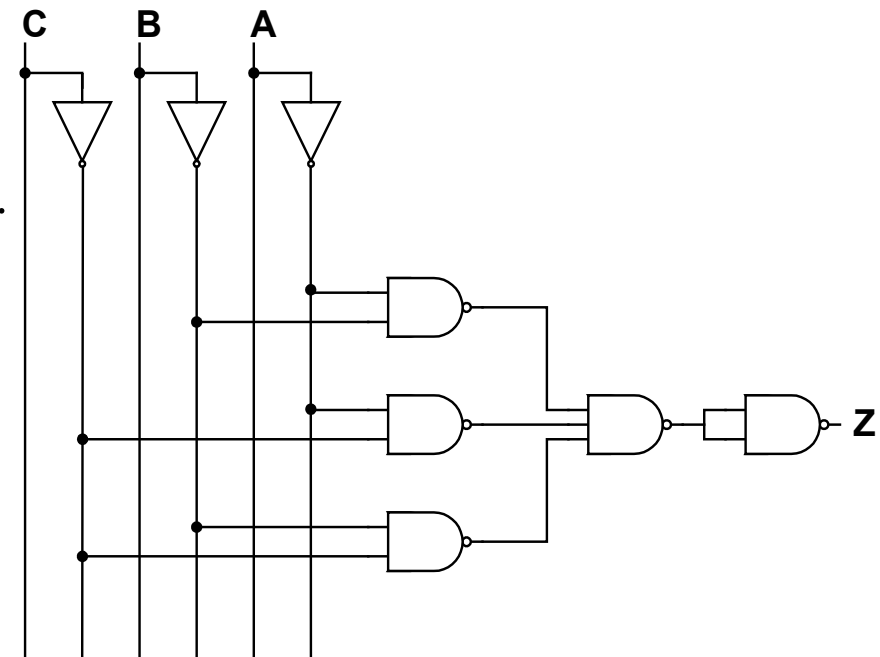
$$Z_{NAF} = \overline{\overline{f(A, B, C)}} = \overline{\overline{(A + B) \cdot (A + C) \cdot (B + C)}} = \dots$$

$$\dots = \overline{\overline{(A + B)} + \overline{\overline{(A + C)}} + \overline{\overline{(B + C)}}} = \dots$$

$$\dots = \overline{\overline{AB} + \overline{AC} + \overline{BC}} = \overline{\overline{AB}} \cdot \overline{\overline{AC}} \cdot \overline{\overline{BC}}.$$

De Morgan:

$$\overline{(A + B)} = \overline{A} \cdot \overline{B}$$



3.5.3 NOR-Formen

- Im folgenden Abschnitt werden so genannte NOR-Formen betrachtet. Sie sind für den *Schaltungsentwurf* von großem Interesse, da mit dieser Konzeption jede beliebige Schaltung mit Hilfe von NOR-Gattern realisierbar wird.
- Prinzipiell können alle *kombinatorischen Schaltungen* entsprechend transformiert werden.
- Die **NOR-Form (NOF)** einer Funktion enthält ausschließlich NOR-verknüpfte Terme. Die NOF kann durch Mehrfachanwendung der De Morgan Regeln aus KDNF, KKNF oder DNF sowie KNF gebildet werden.
- Die Umsetzung in eine NOF geschieht wie folgt:

3.5.3 NOR-Formen

- Eingangsfunktion: **KDNF** oder **DNF** $f_D(\mathbf{x})$
 - Viermalige Negation: $\overline{\overline{\overline{\overline{f_D(\mathbf{x})}}}}$ führt auf die gesuchte NOF.
 - Das Ergebnis lautet: $Z_{NOF} = \overline{\overline{\overline{\overline{f_D(\mathbf{x})}}}}$.

- Eingangsfunktion: **KKNF** oder **KNF** $f_K(\mathbf{x})$
 - Zweimalige Negation: $\overline{\overline{f_K(\mathbf{x})}}$ führt auf die gesuchte NOF.
 - Das Ergebnis lautet: $Z_{NOF} = \overline{\overline{f_K(\mathbf{x})}}$.